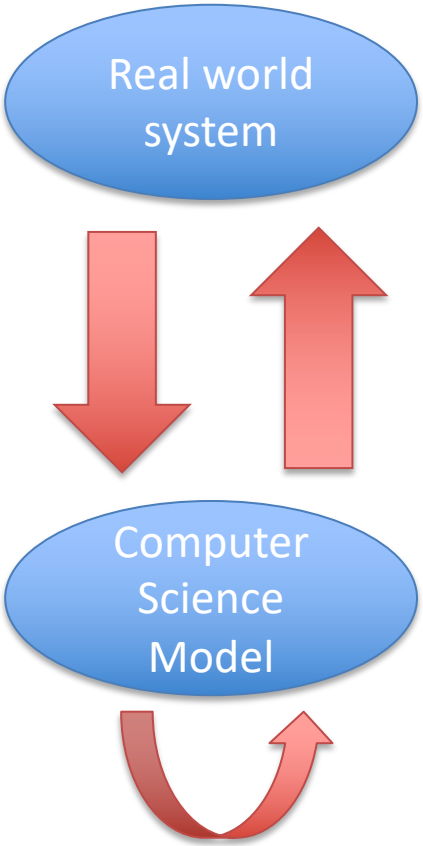
1.

RL1 - DETERMINISTIC FINITE AUTOMATA (DFA)

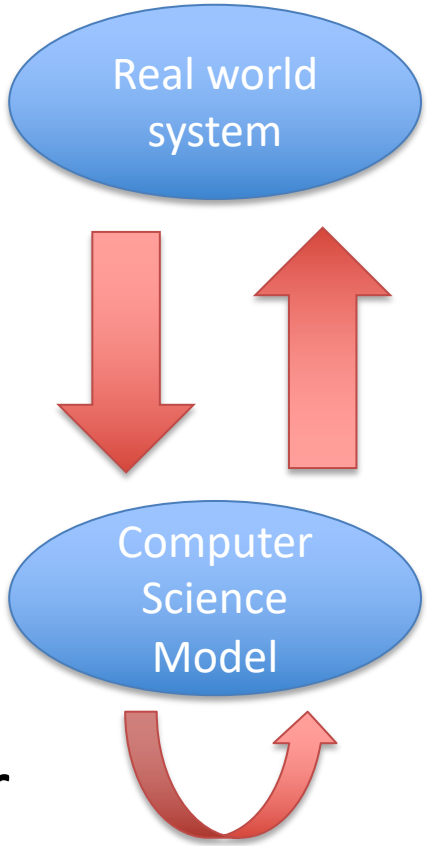
Computer Science models

- Computer science models describe some important real world behaviour
- They do so by abstracting, simplifying and codifying
- to the point where subtle analyses and observations and conclusions can be made
- and related back in a meaningful way to the physical world thereby shedding light on what was not obvious before



The role of CS models

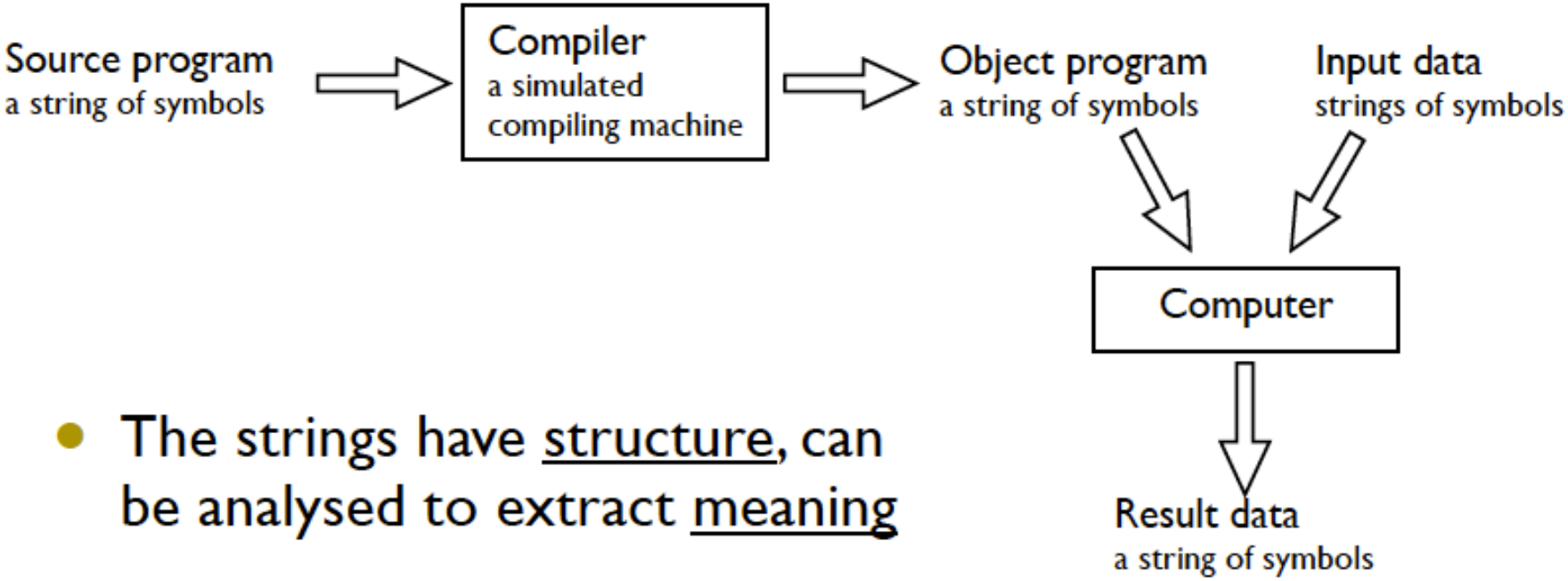
- To focus on selected aspects of systems/programs
- To structure the design of systems/programs
- To analyse the design of systems/programs
  - Correctness => safety by design
  - Security => privacy by design
  - Performance => energy saving by design
  - …
- Often the models give rise to tools for automatic solution for subsystems



Computer science models

- Added benefit: computer science models have many practical applications
  - Compilers: scanning, parsing, translation, …
  - Text search: editors, file systems, the web
  - Software verification: do programs behave as expected?
  - IDEs: model-driven software development
  - Security: enforcement of security policies
  - …

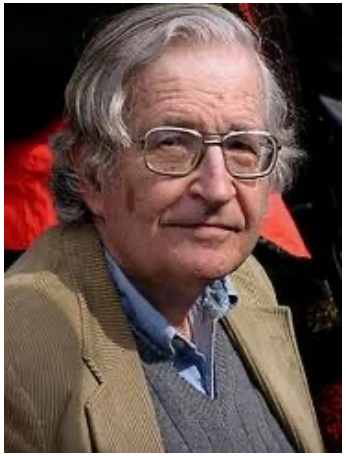
Example: A compiler



- The strings have <u>structure</u>, can be analysed to extract <u>meaning</u>
- Each string is an element of some "language"

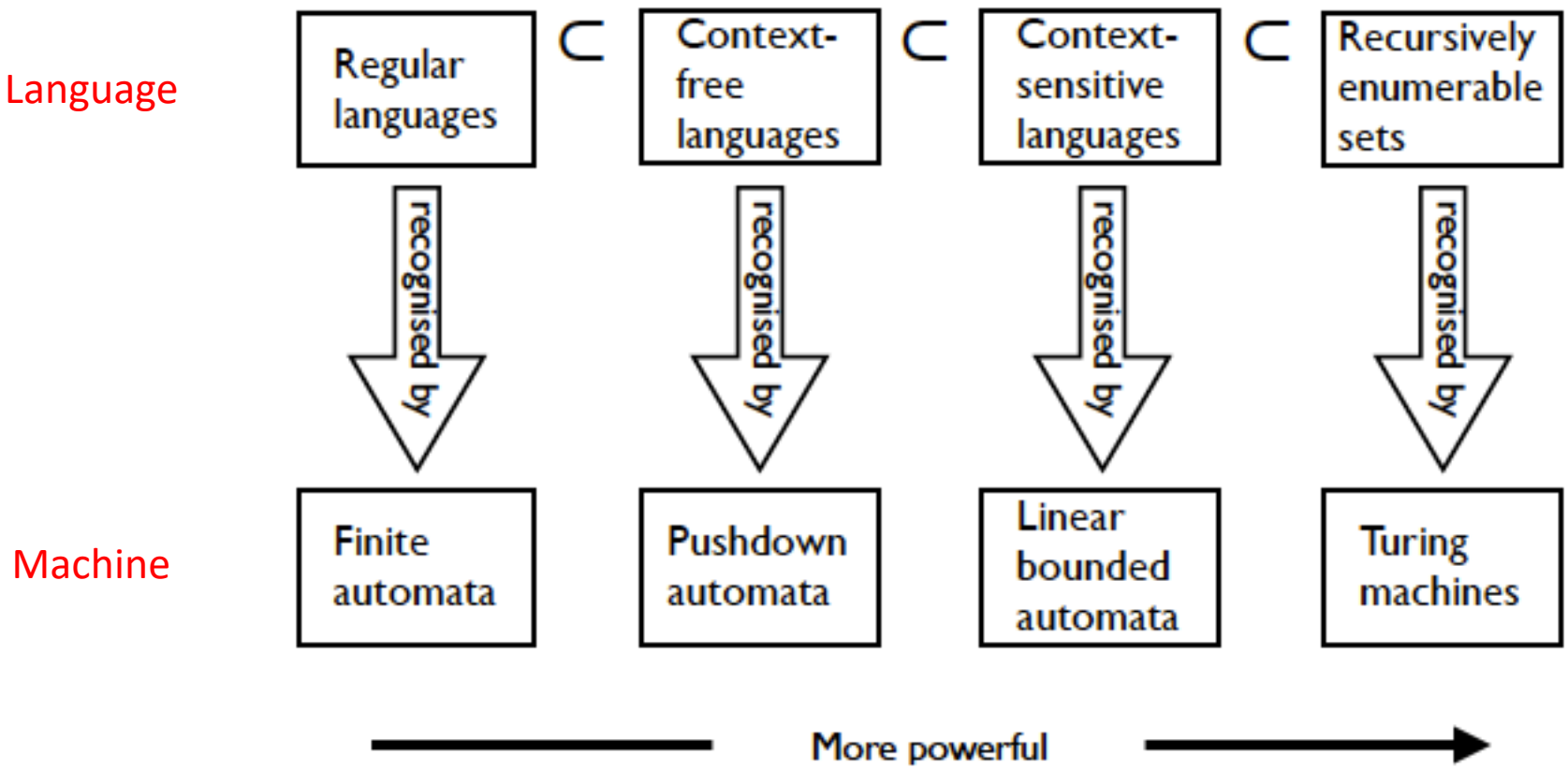
A strong influence from Linguistics

Two strands of work pulled together



- **Chomsky**: the languages humans invented for communication with one another and the languages needed for humans to communicate with machines share many basic principles
- **Turing**: if some human can figure out an algorithm for solving a particular class of problems then the machine can be told to follow the steps and execute the algorithm exactly and precisely

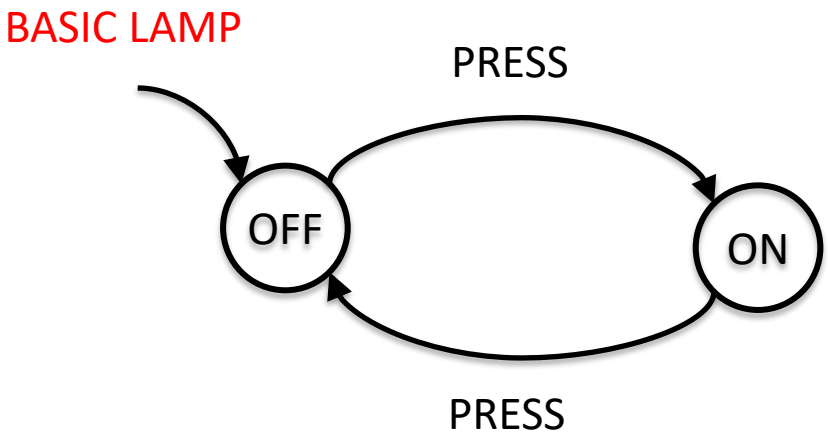
Models have different expressive power



The Chomsky Hierarchy

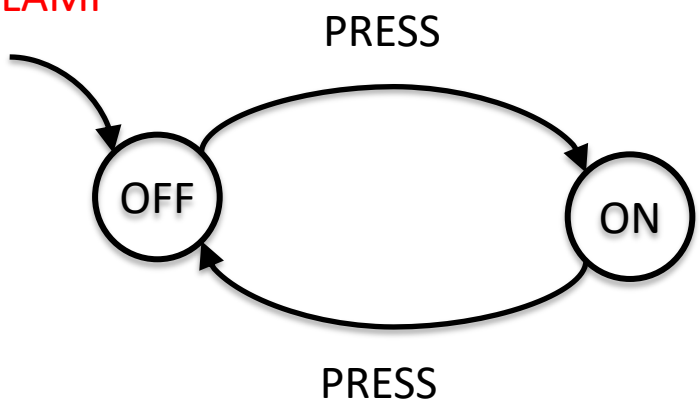
MODELLING WITH DFA

Example: A lamp

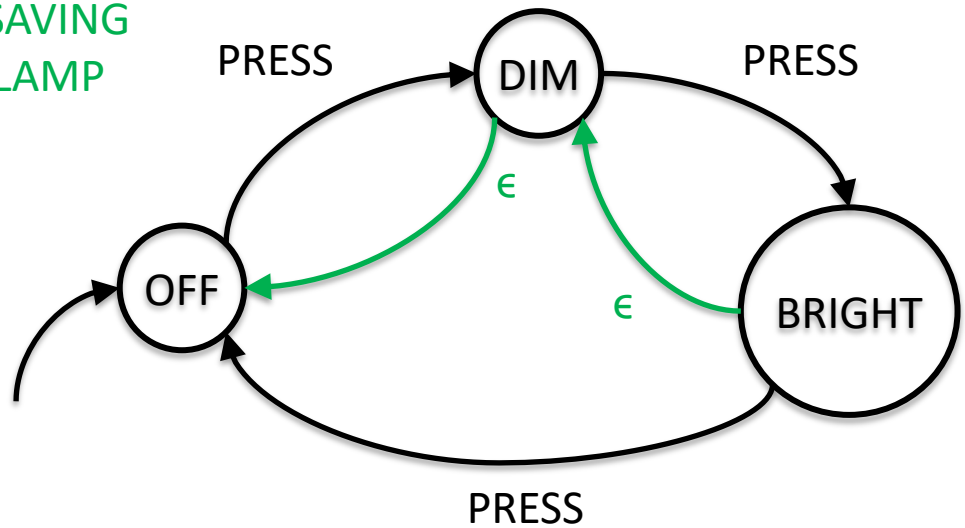


Example: A lamp

BASIC LAMP



ENERGY SAVING
SMART LAMP



ϵ are ”internal actions”

Example: A vending machine

- Coffee: 15 kr
- Coins accepted: 5 kr
- Max capacity: 15 kr

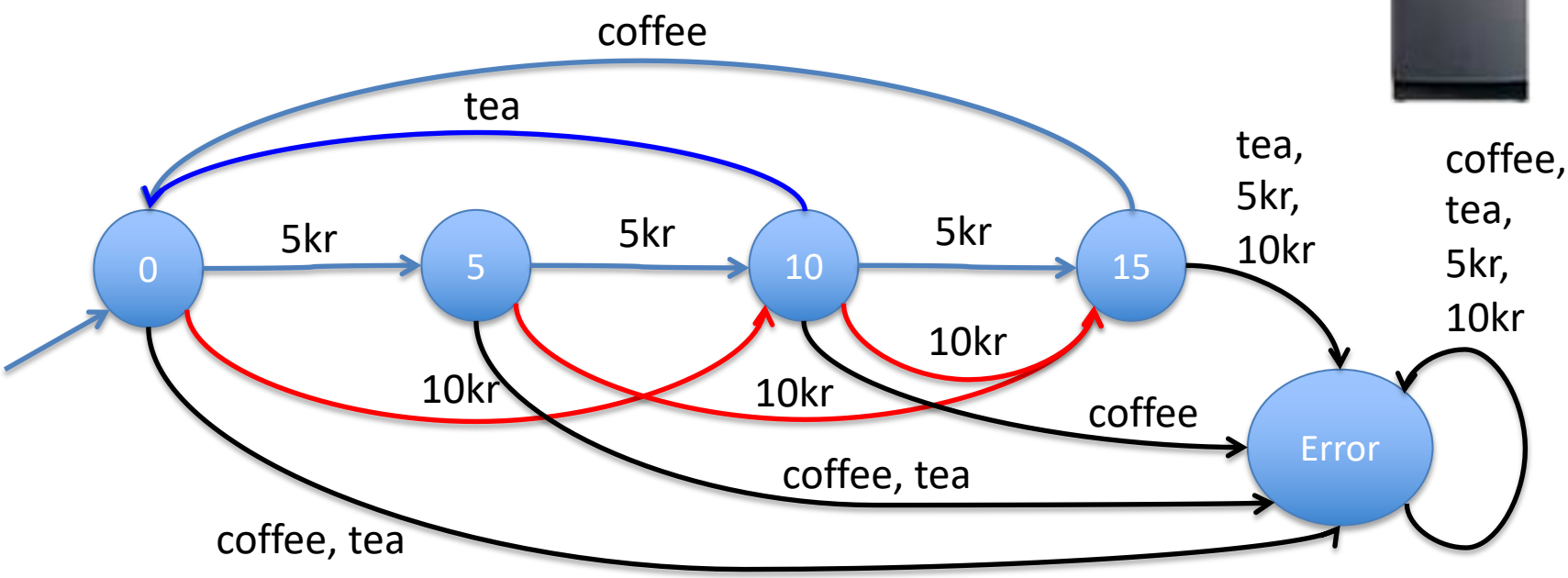
- Also tea: 10 kr
- Also accept: 10 kr

- What happens if we don't follow "the rules"?

- Can we model that we give change back?

Example: A vending machine

A possible model

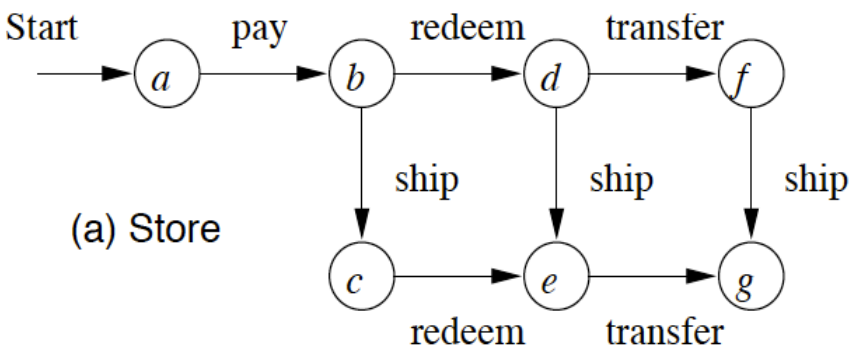


Example from HMU: E-commerce

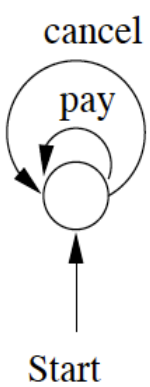
- Three principals: Customer, Store and Bank
- Model how
  - Customer pays money to Store
  - Customer cancels payment by notifying Bank
  - Store ships goods to Customer
  - Store redeems money from Bank
  - Bank transfers money to Store

Example: E-commerce

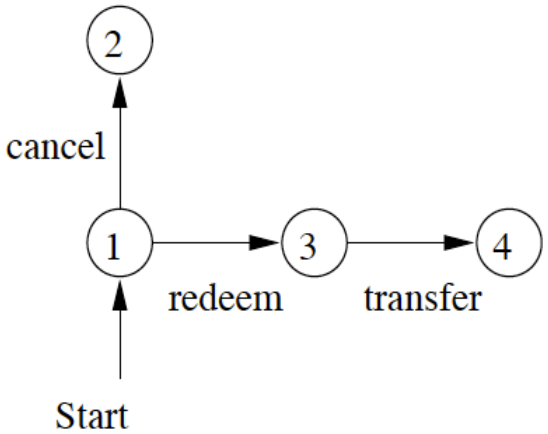
Finite automata
for each principal



(a) Store



(b) Customer

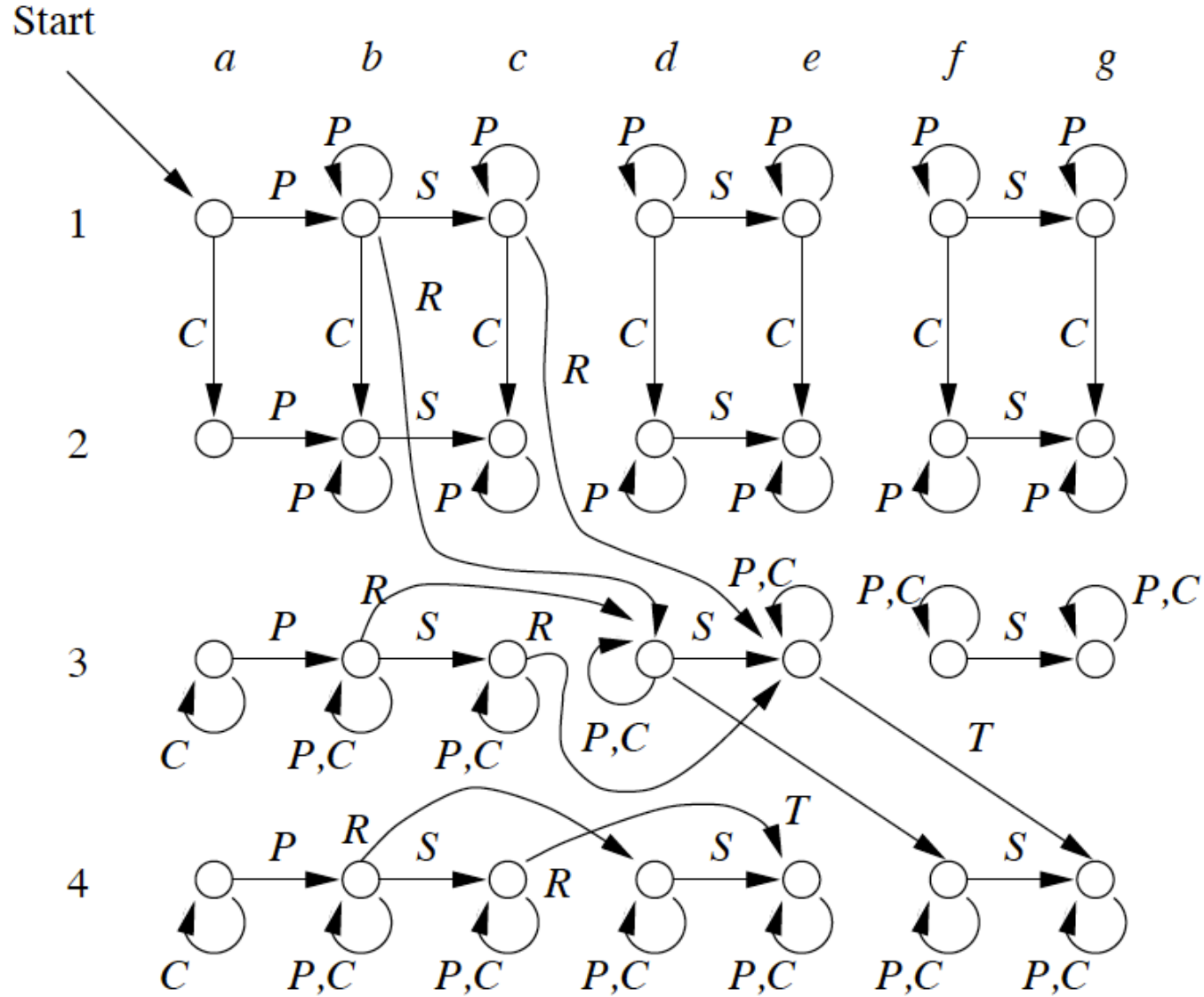


(c) Bank

Example: E-commerce

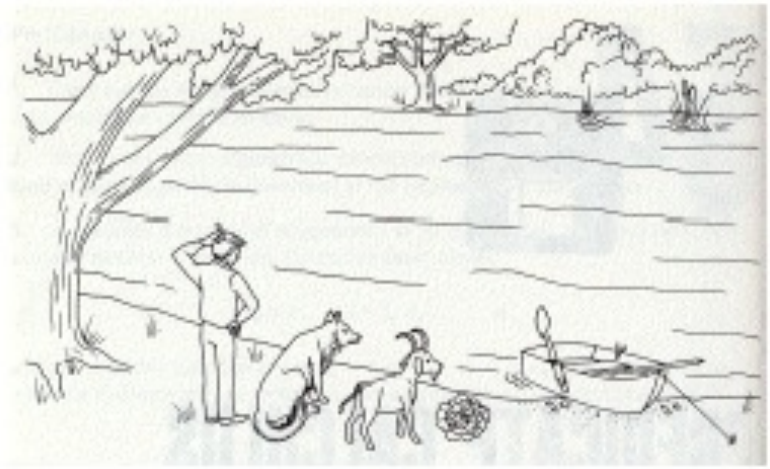
The combined finite automaton

Product automaton: Keep track of the state of all three automata while synchronising on joint actions



P: pay
S: ship
C: cancel
R: redeem
T: transfer

More examples: games & puzzles



ALPHABETS, STRINGS AND LANGUAGES

Key concepts

- Σ: alphabet – the actions of the automaton
- Strings are sequences of symbols from an alphabet :
  - $w = a_1\ a_2 \ldots a_k$: a finite sequence of symbols a_i from Σ of length k (≥ 0)
  - $w = \varepsilon$: the empty sequence (of length 0)
- we write $|w|$ for the length of the string w

Key concepts

- If x and y are strings, then xy denotes the concatenation of x and y
  - Concatenation is associative $(xy)z = x(yz)$
  - ε is the identity: $\varepsilon w = w\varepsilon = w$
- The lengths add up: $|xy| = |x|+|y|$
- If w is a string we define w^n to be the concatenation of n copies of w:
  - *basis*: $w^0 = \varepsilon$
  - *induction*: $w^{i+1} = w^i w$ for $i \geq 0$

Sets of strings and languages

- Σ^k: the set of strings of length *k*
- Σ^*: The set of all strings over Σ (with finite length)
  - $\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^2 \cup \ldots$
- Σ^+: The set of non-empty (finite) strings over Σ
  - $\Sigma^+ = \Sigma^1 \cup \Sigma^2 \cup \ldots$

- A language L is a subset of Σ*
  - Two special cases:
    - The empty language: Ø
    - The language of the empty string: {ε}

Different languages!

Specifying languages

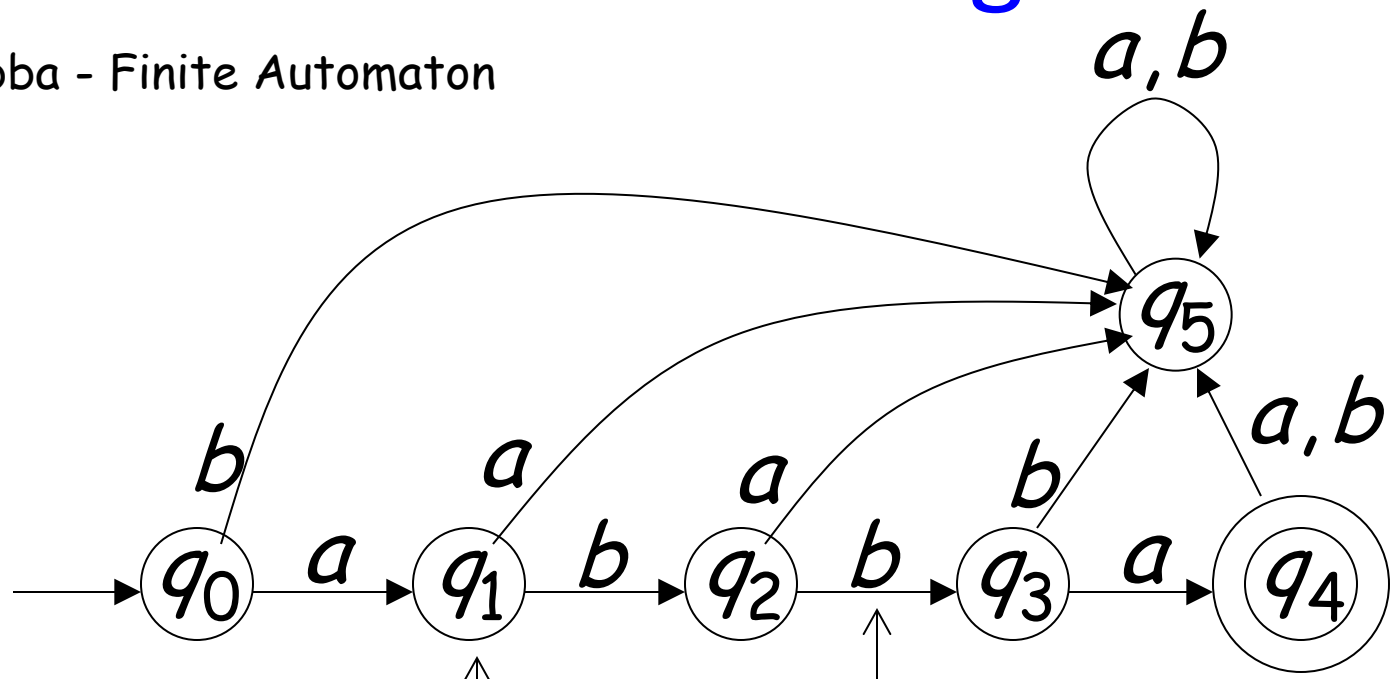
- How is the language specified?
  - Deterministic finite automaton (RL1, today)
  - Non-deterministic finite automaton (RL2)
  - Regular expression (RL3)

- We shall show that the three methods are equivalent!
- They all specify regular languages

DFA: DEFINITION AND REPRESENTATIONS

Transition Diagram

abba - Finite Automaton



initial state

final state "accept"

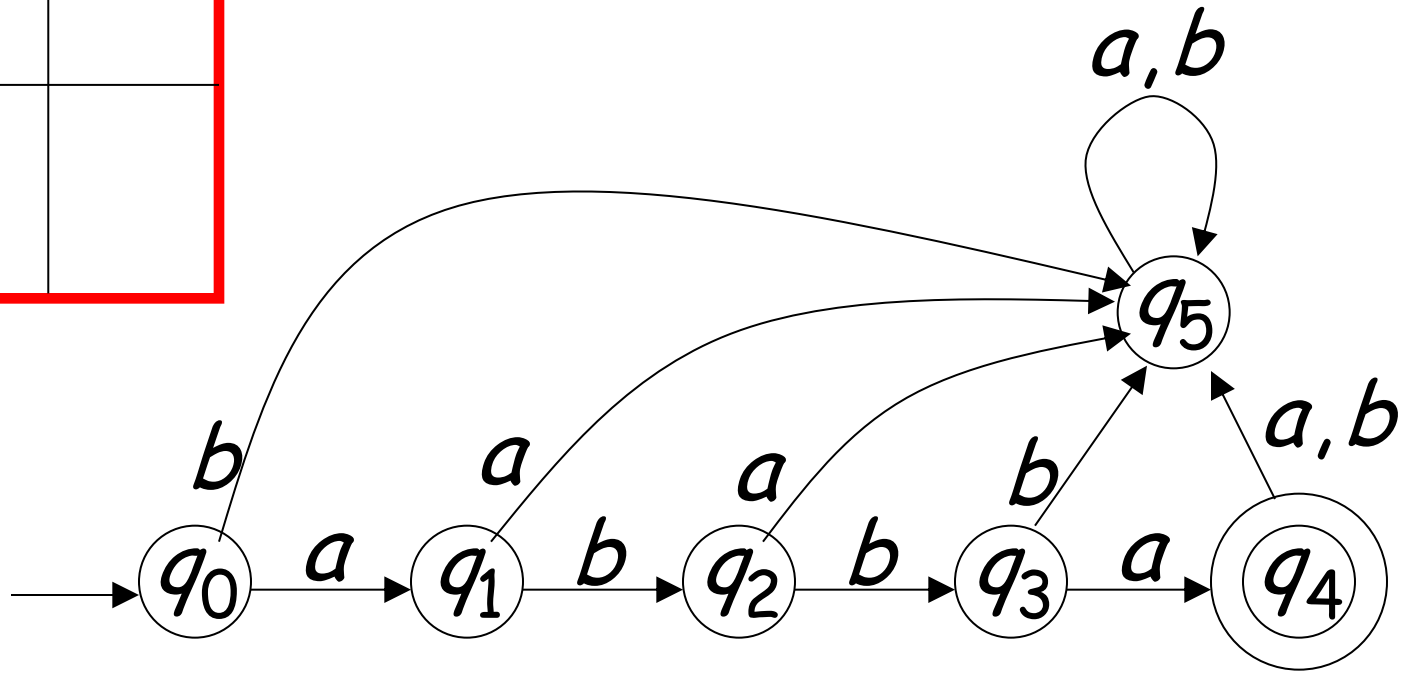
transition

state

$\Sigma=\{a,b\}$

Transition Table

	δ	a	b
→	q_0	q_1	q_5
	q_1	q_5	
	q_2		
	q_3		
	q_4		
*	q_5		



Formal definition

A *deterministic finite automaton* consists of

- Q: a finite set of states
- Σ: a finite set of input symbols, an alphabet
- $\delta: Q \times \Sigma \rightarrow Q$: a transition function
  for each state q in Q and each symbol a in Σ it determines a new state δ(q,a) in Q
- q_0: the initial state; an element of Q
- F: the final states; a subset of Q

Often written as $A = (Q, \Sigma, \delta, q_0, F)$

Formal definition

$A = (Q, \Sigma, \delta, q_0, F)$

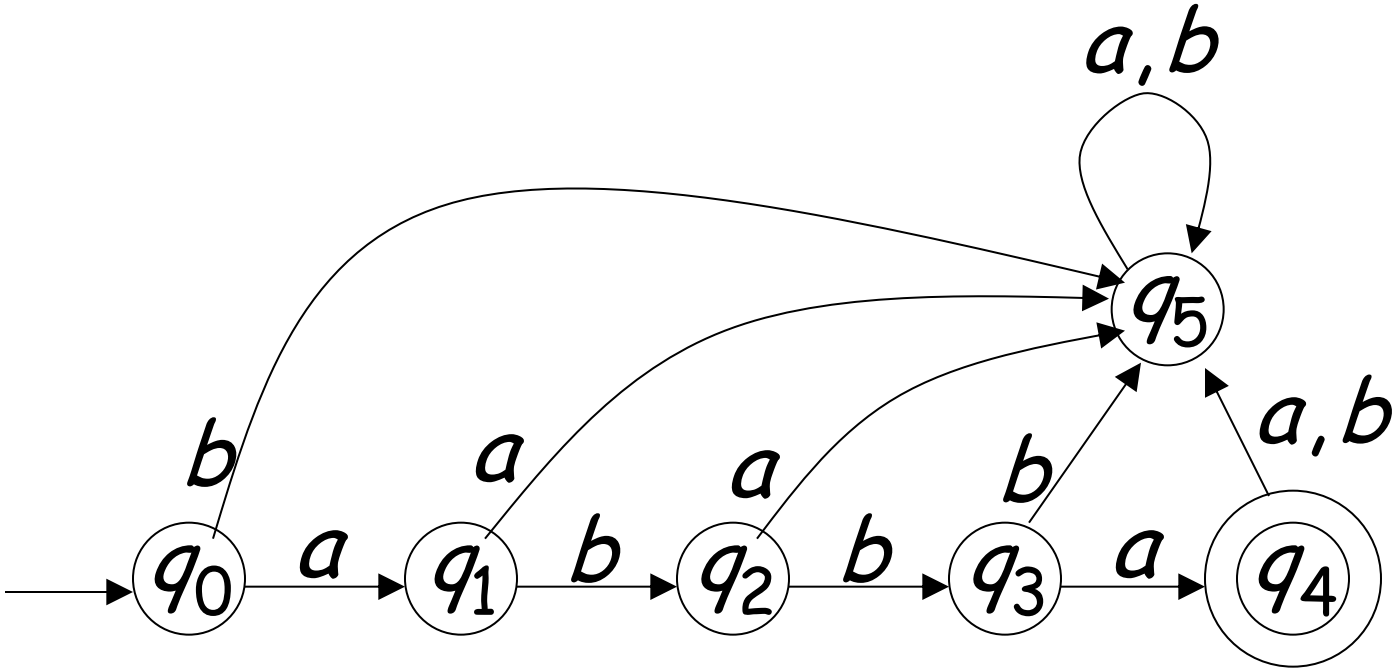
$Q =$

$\Sigma =$

$\delta =$

$q_0 =$

$F =$



Let's do it!

Exercise 2.2.4 (b)

- Consider the alphabet {0,1} and specify a DFA that accepts the set of all strings with three consecutive 0's (not necessarily at the end).

- (We will do the rest of exercise 2.2.4 later)

THE LANGUAGE OF A DFA

How does a DFA define a language?

Consider a string w

- Start in the initial state q_0
- Read the first symbol, say a_1, of w
- Determine the new state $q_1 = \delta(q_0,a_1)$
- Read the second symbol, say a_2, of w
- Determine the new state $q_2 = \delta(q_1,a_2)$
- …
- Let q_k be the state obtained after having read the last symbol of w
- If q_k is in F then accept w; otherwise reject w

Formalisation

By induction on the length of the string w define:

- $\delta^*(q,\varepsilon) = q$
- $\delta^*(q,aw) = \delta^*(\delta(q,a),w)$

  or alternatively $\delta^*(q,wa) = \delta(\delta^*(q,w),a$

It can be shown that $\delta^*(q,xy) = \delta^*(\delta^*(q,x),y))$

NOTE: It's one of the exercises ☺

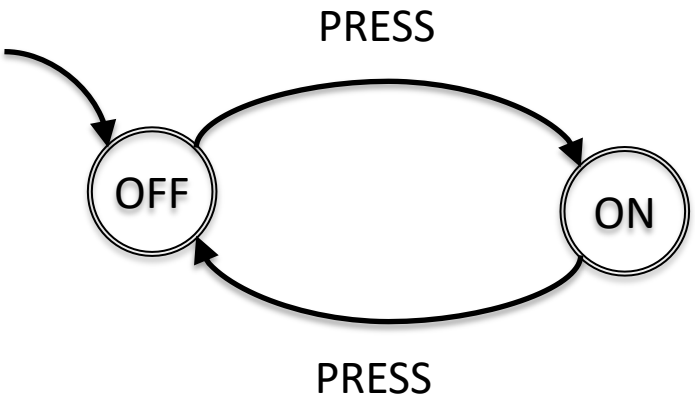
The language of $A = (Q, \Sigma, \delta, q_0, F)$

$L(A) = \{w \mid \delta^*(q_0,w) \text{ is in } F\}$

We say that *A accepts w* if w is in L(A)

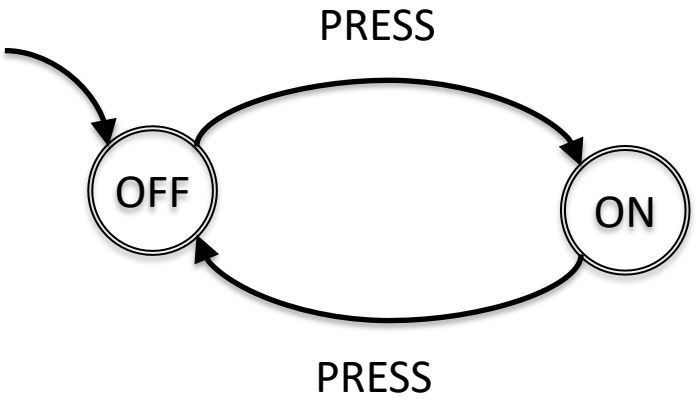
Try Out

Implement the basic lamp example in your favourite programming language



Implement δ as a function.
Implement δ* as a recursive function.

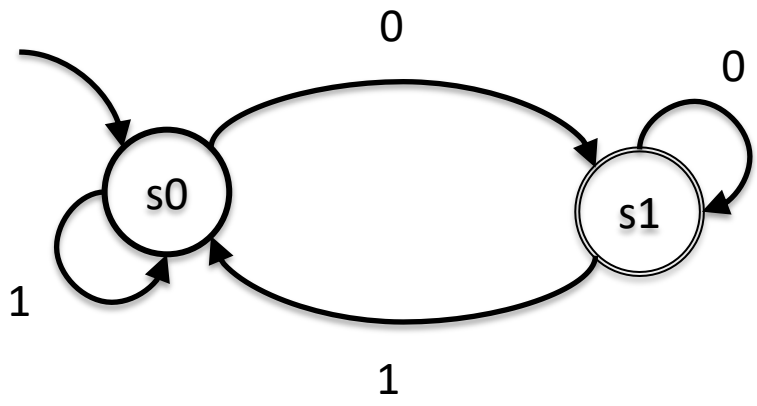
Try Out




```
type State = ON | OFF
type Symbol = PRESS

let delta state symbol =
   match state , symbol with
   | ON , PRESS -> OFF
   | OFF , PRESS -> ON

let rec deltastar state input =
   match input with
   | [] -> state
   | a::w -> deltastar (delta state a) w

printfn "Finished at state %A" (deltastar ON (PRESS::PRESS::[]))
```

Showing correctness of a DFA

Consider the alphabet {0,1} and the language of of all strings ending with 0. Has the following DFA the same language?



DECISION PROBLEMS FOR REGULAR LANGUAGES

Three fundamental problems

- The membership problem:

  Given a language L and a string w, does w belong to L?

- The emptiness problem:

  Is the language L empty?

- The equivalence problem:

  Are the two languages L and M equal?

Three fundamental problems

- The membership problem:

  L1 = all strings that contain at least one occurrence of COFFEE preceded by PRESS.

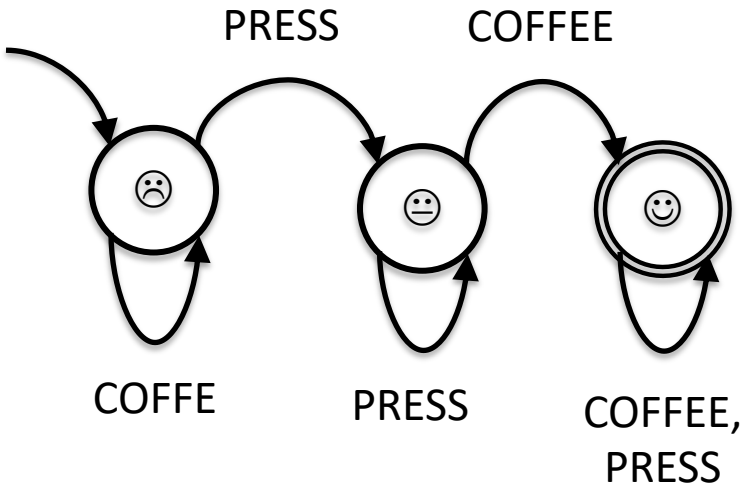
  L2 = all strings that lead the automaton below to state ☺.

  L3 = all strings that lead the automaton below to state ☺ in just 1 step.

  Does PRESS,PRESS,PRESS,COFFEE belong to L1? And to L2? And to L3?
- The emptiness problem:

  Is the language L1 empty? And L2? And L3?
- The equivalence problem:

  Are L1 and L2 equal?



COMPOSING LANGUAGES AND DFA

Good news!

- The union of two regular languages is regular
- The intersection of two regular languages is regular
- The complement of a regular language is regular
- The difference between two regular languages is regular
- The reversal of a regular language is regular
- The Kleene closure (star) of a regular language is regular
- The concatenation of two regular languages is regular

Composition operations

How are those composition operations defined?

How can we compute those operations?

We will see most of them in the next lectures:

- NFA are ”easier” to compose
- Regular expressions are indeed a language for composing regular languages
- The conversion of NFA to regular expressions uses indeed many of those operations

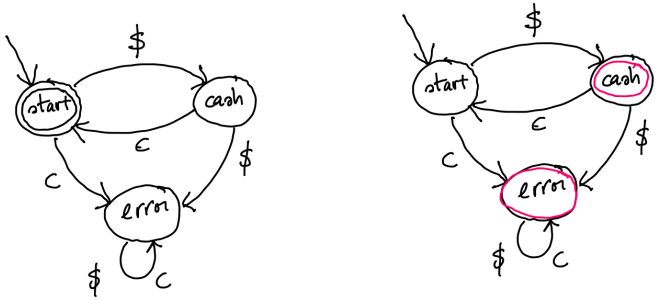
We will see today some examples.

Example: complement

$$L_1 \rightarrow \overline{L_1} = \{ w \mid w \notin L_1 \}$$

$$L(A) \rightarrow \overline{L(A)} = L(\overline{A})$$

$$\overline{A} = \langle Q, \Sigma, \delta, q_0, Q \setminus F \rangle$$



If L1 is the language of “good” things, then its complement is the language of the “bad” things”!

Examples: good/bad interactions, valid/invalid keywords, good/bad logs…

Example: intersection

Given two automata sharing an alphabet

$$A_1 = \langle Q_1, \Sigma, \delta_1, q_0^1, F_1 \rangle$$

$$A_2 = \langle Q_2, \Sigma, \delta_2, q_0^2, F_2 \rangle$$

We can define their intersection as follows:

$$L(A_1) \cap L(A_2) = L(A_1 \cap A_2)$$

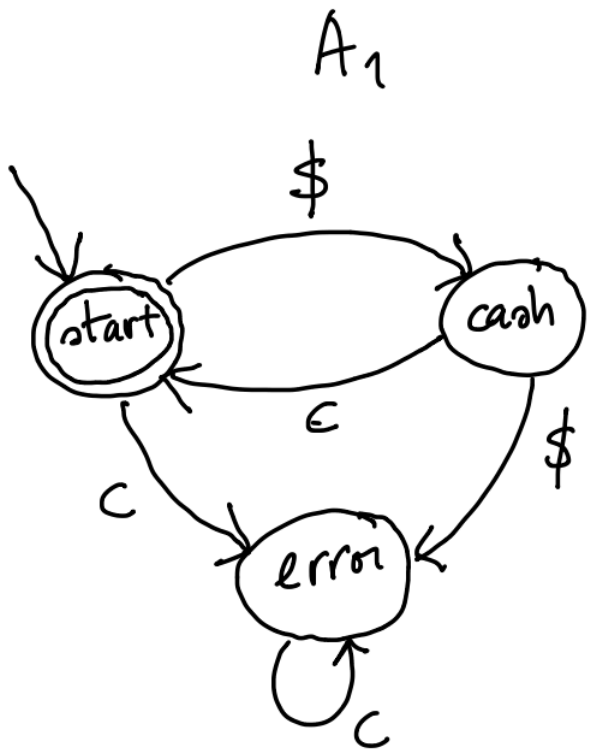
$$A_1 \cap A_2 = \langle Q_1 \times Q_2, \Sigma, \delta, (q_0^1, q_0^2), F_1 \times F_2 \rangle$$

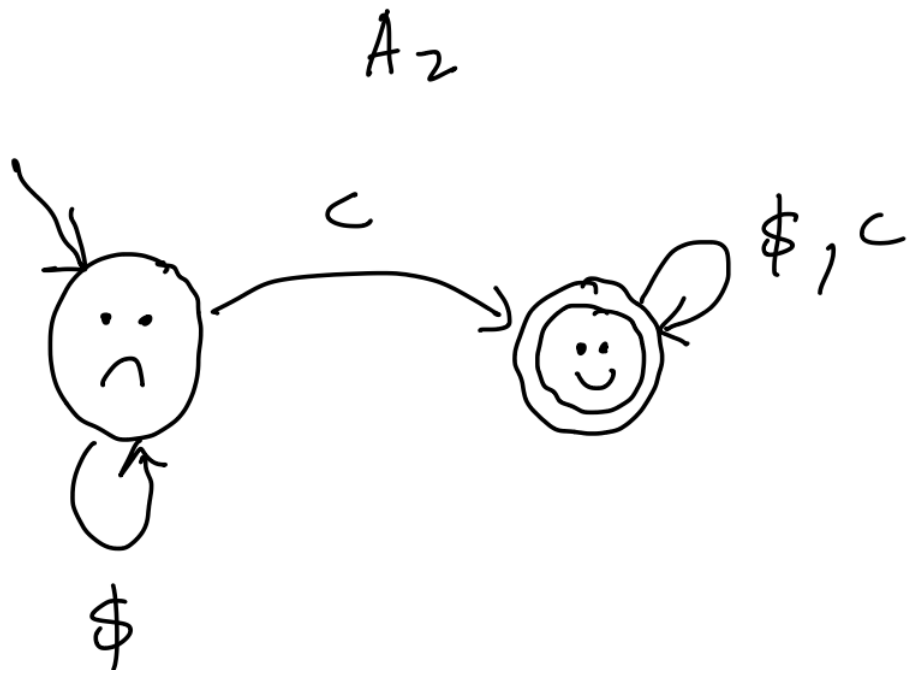
$$\delta((q_1, q_2), a) = (\delta_1(q_1, a), \delta_2(q_2, a))$$

Intersection can model synchronization of components! Example (see next slide)

Try out

Compute the intersection of these two automata:





SOLVING DECISION PROBLEMS

Decision Problems

How would you solve the fundamental problems for languages using DFA?

- Does w belong to L1?
- Is L1 empty?
- L1=L2?

Deciding Membership

Does w belong to L1?

Deciding Membership

Does w belong to L1?

Solution:

- If L1 is given as a DFA, we just check if $\delta^*(q_0,w)$ is in F.
- Otherwise:
  - Design a DFA A for L1.
  - Show that the L(A) = L1.
  - Apply the above method.

Deciding Emptyness

Is L1 empty?

Deciding Emptyness

Is L1 empty?

Solution:

- If L1 is given as a DFA, we can analyze the transition diagram to see if there is any state in F that is reachable from the initial state.
- Otherwise:
  - Design a DFA A for L1.
  - Show that the L(A) = L1.
  - Apply the above method.

Deciding Equality

L1=L2?

Deciding Equality

L1=L2?

Solution:

- If L1 and L2 are given as a DFA, we can apply the equality test algorithm of HMU 4.4.2
- Otherwise:
  - Design DFA A1 for L1, and A2 for L2
  - Show that the L(A2) = L1 and L(A2)=L2.
  - Apply the above method.

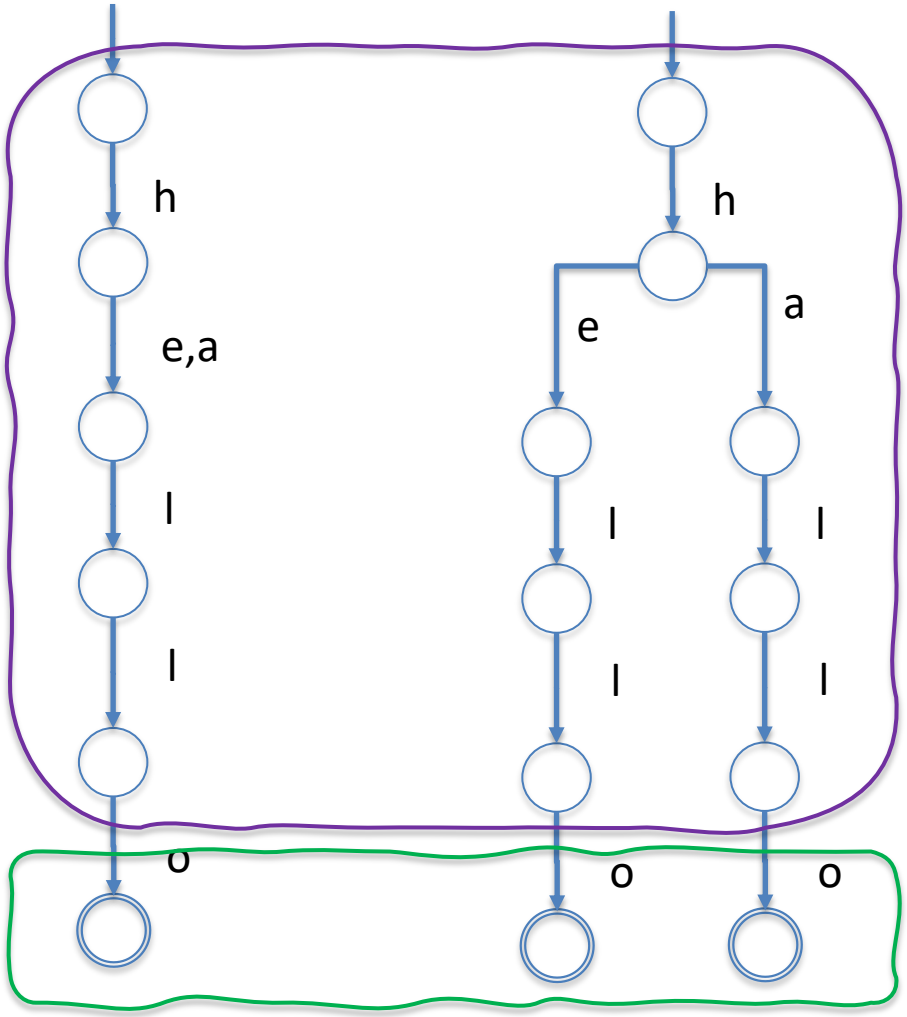
Deciding Equality of DFA

Equality Game Algorithm:

- Initial state: form two groups:
  - all accepting states (of both DFA) together;
  - all other states (of both DFA) together.
- A state s in group G that has a transition with label x to a state in a group G' can challenge a state s' in G: can you move to group G'?
  - State s wins the challenge if s' cannot reply with a transition to a state in G'. In that case we split G in two two subgroups:
    - States that can reply to the challenge
    - States that cannot reply to the challenge
- When there are no more challenges that would split groups, the game stops.
- The DFA are equivalent iff their initial states are in the same group.

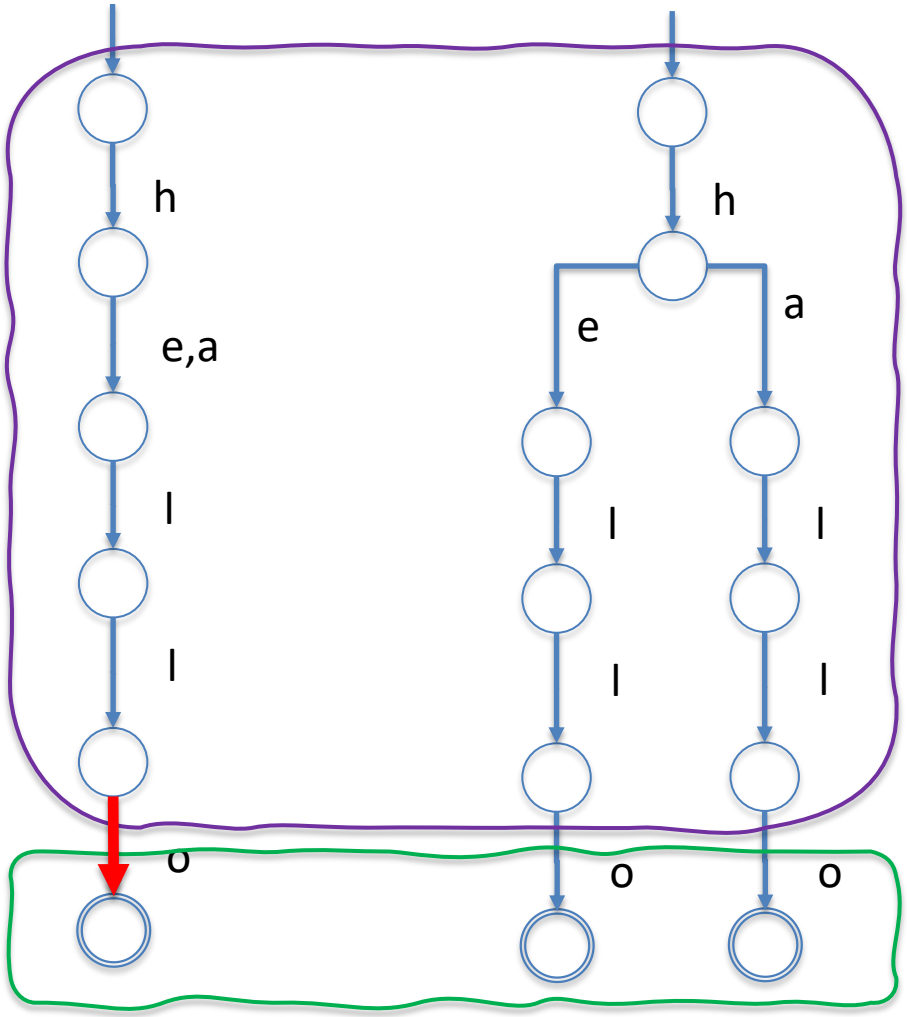
Let's try!

Deciding Equality of two DFA for the language {hello,hallo}



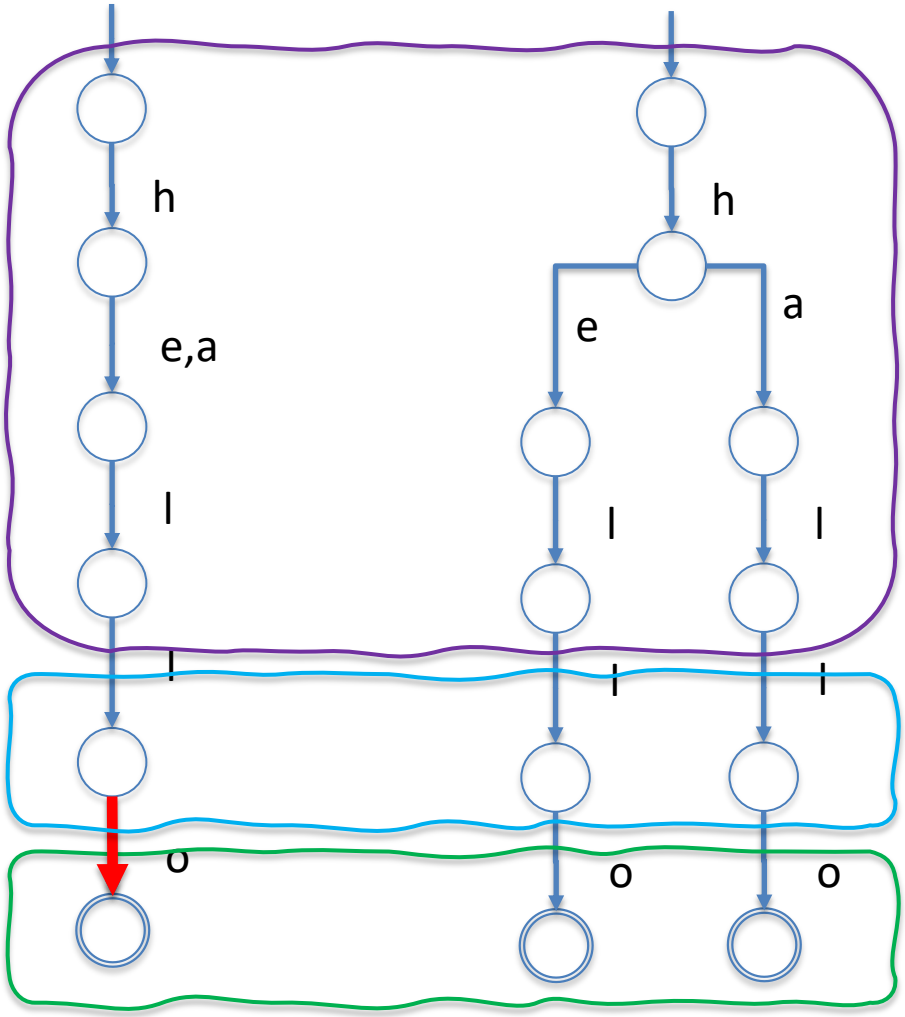
Initial state of the algorithm

Deciding Equality of two DFA for the language {hello,hallo}



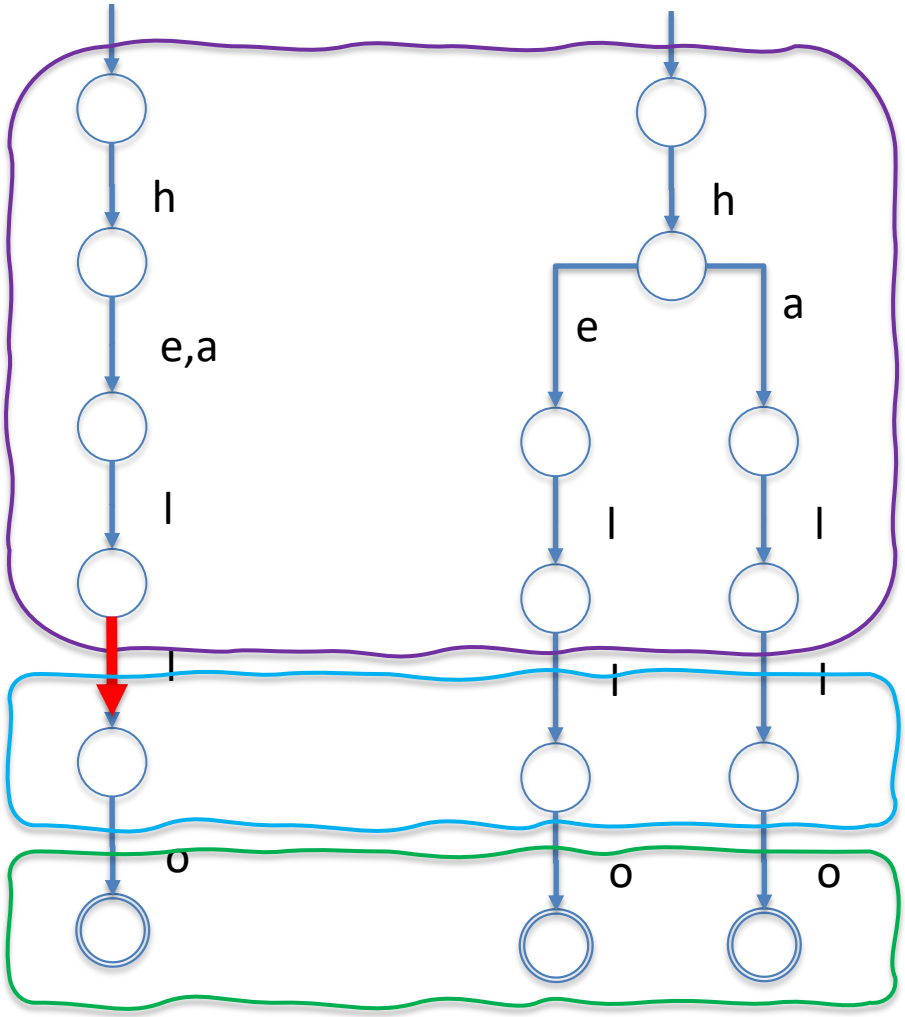
Challenge indicated as

Deciding Equality of two DFA for the language {hello,hallo}



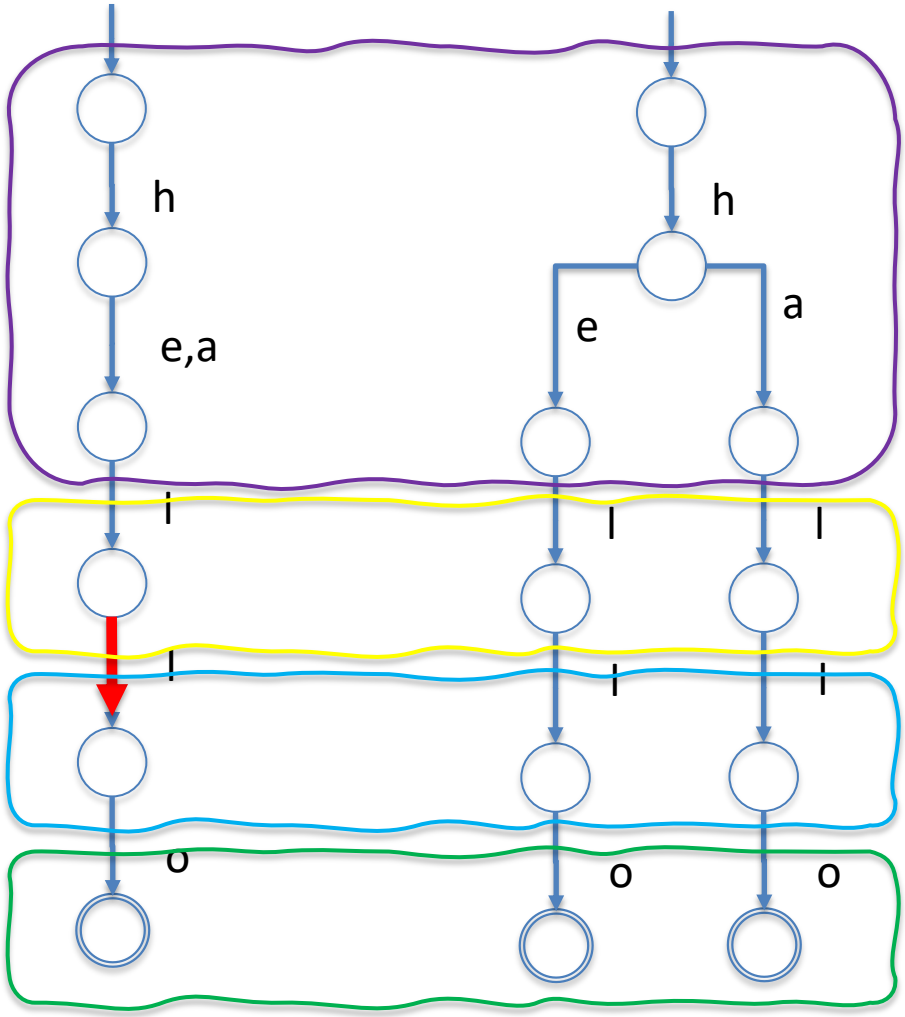
Spliting according to challenge

Deciding Equality of two DFA for the language {hello,hallo}



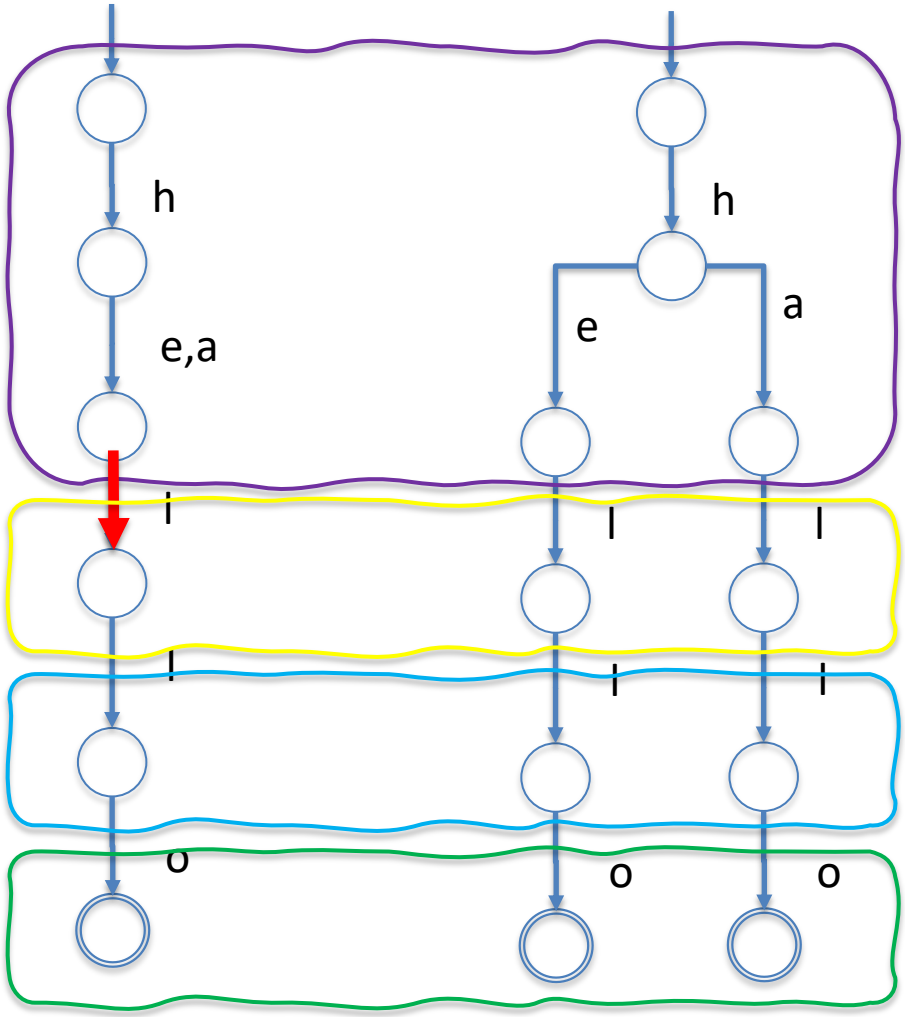
Challenge indicated as

Deciding Equality of two DFA for the language {hello,hallo}



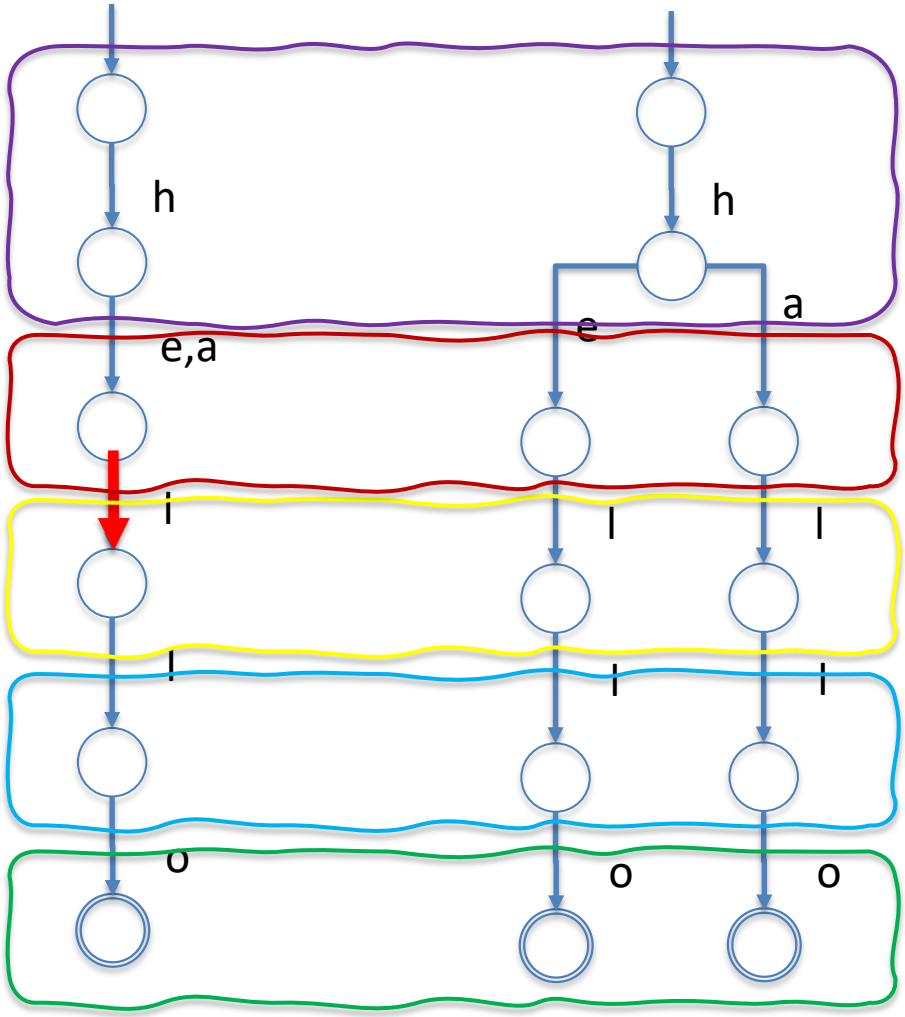
Spliting according to challenge

Deciding Equality of two DFA for the language {hello,hallo}



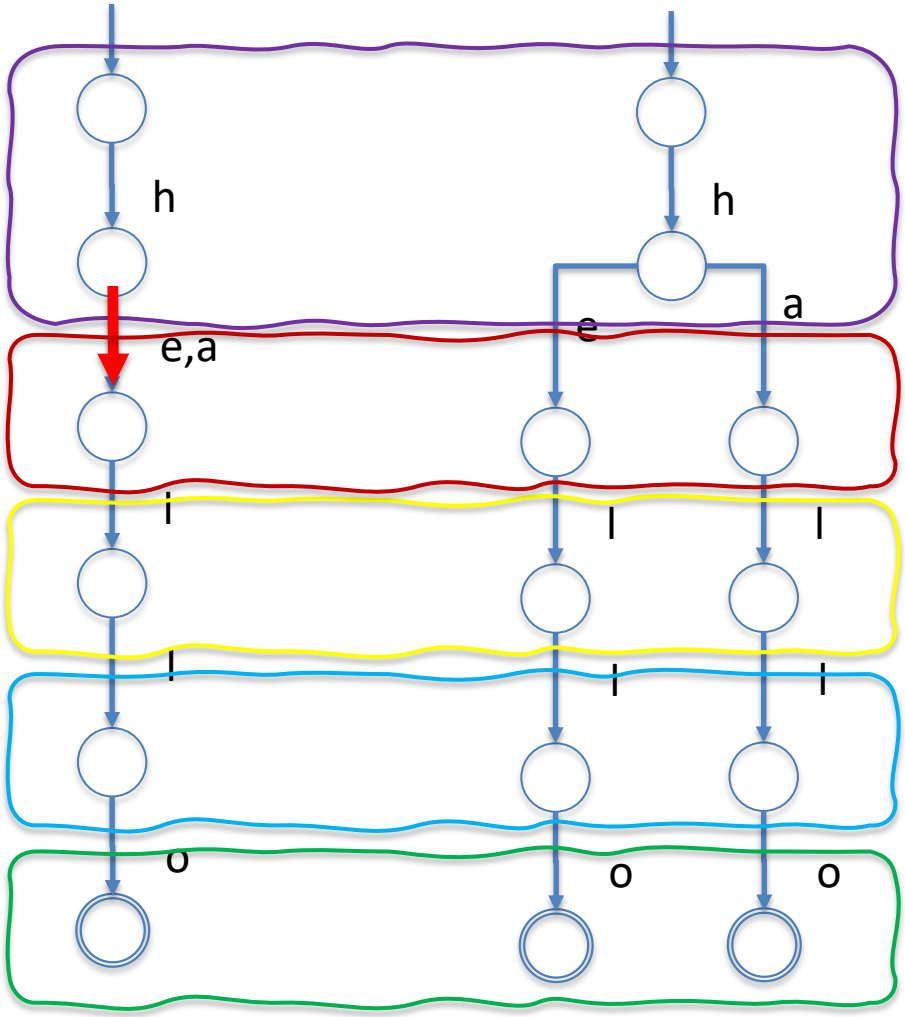
Challenge indicated as

Deciding Equality of two DFA for the language {hello,hallo}



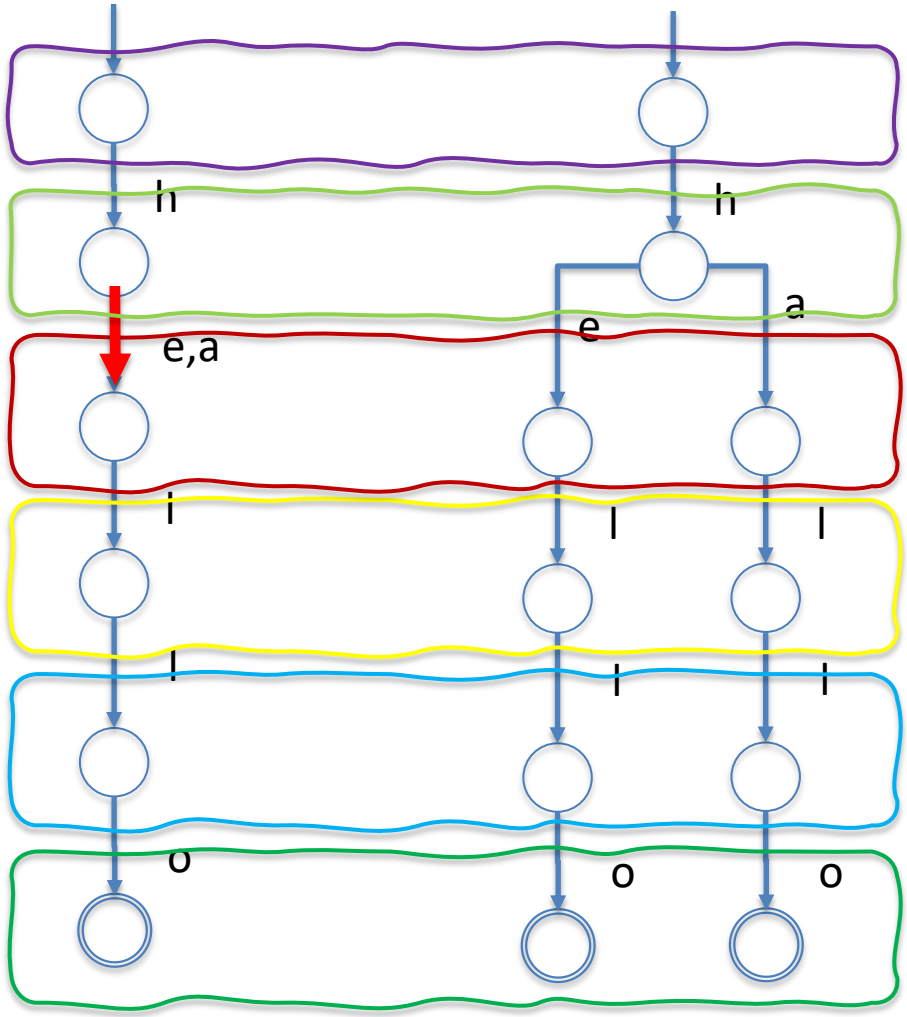
Spliting according to challenge

Deciding Equality of two DFA for the language {hello,hallo}



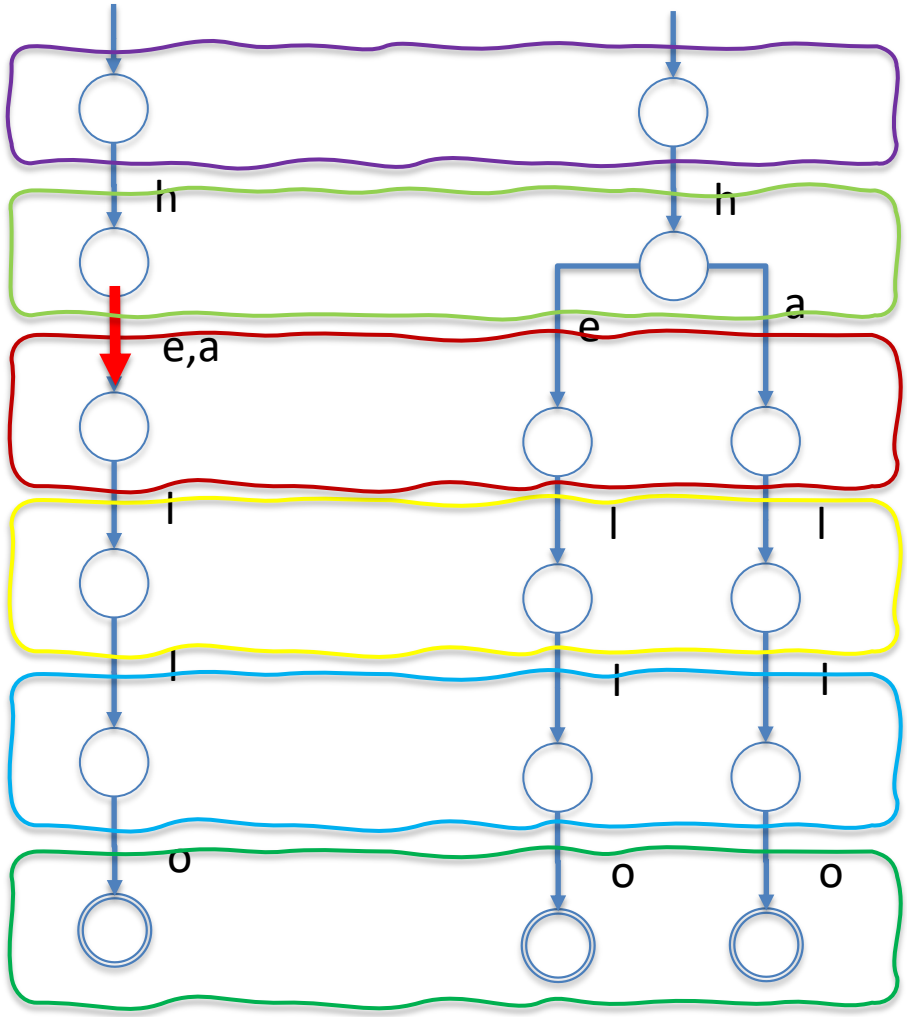
Challenge indicated as

Deciding Equality of two DFA for the language {hello,hallo}



Spliting according to challenge

Deciding Equality of two DFA for the language {hello,hallo}

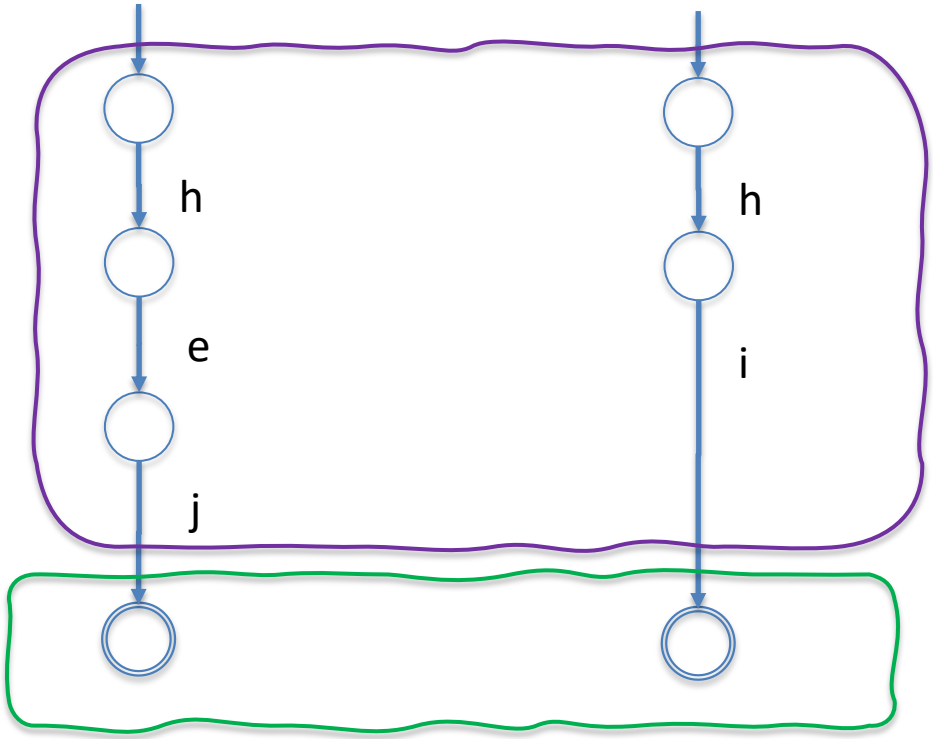


No more challenges exist!

Initial states are in the same group

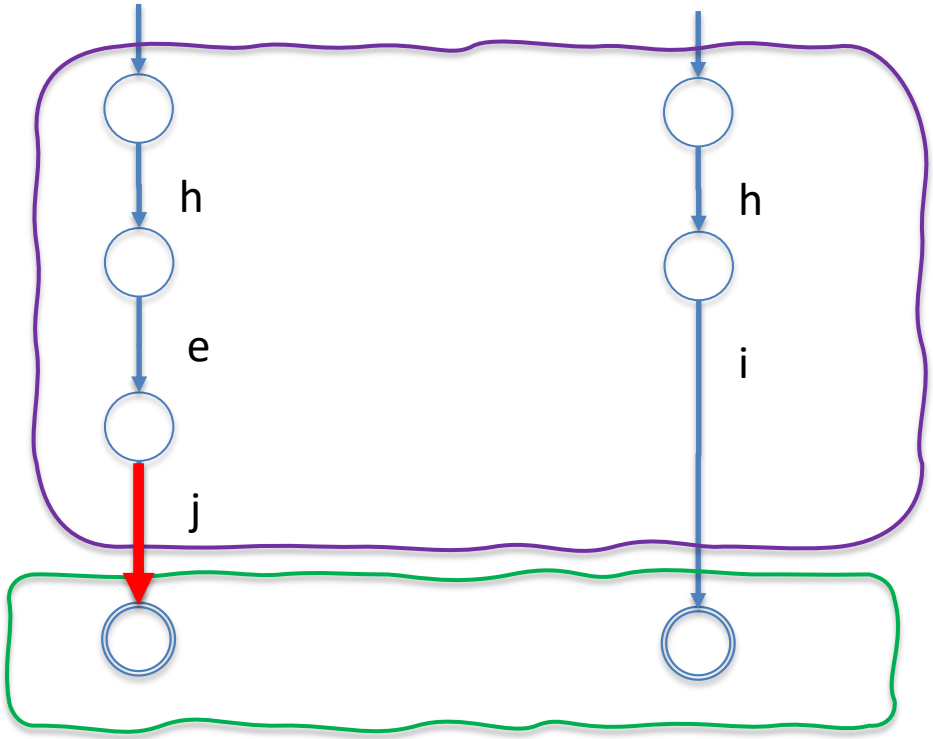
Therefore the DFA are equivalent

Deciding (non) Equality of two DFA



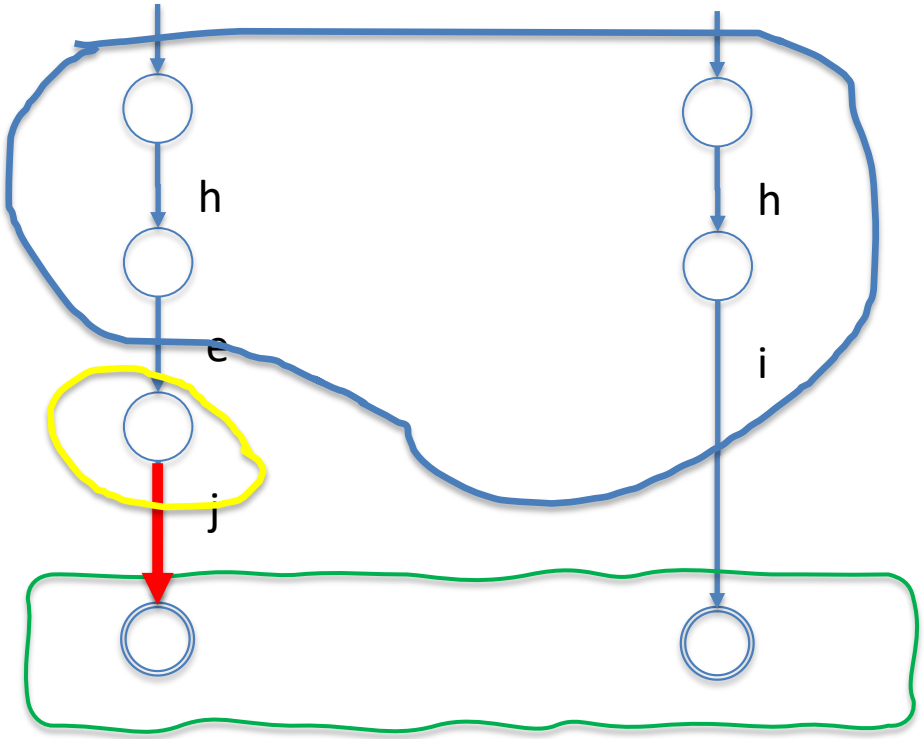
Initial state of the algorithm

Deciding (non) Equality of two DFA



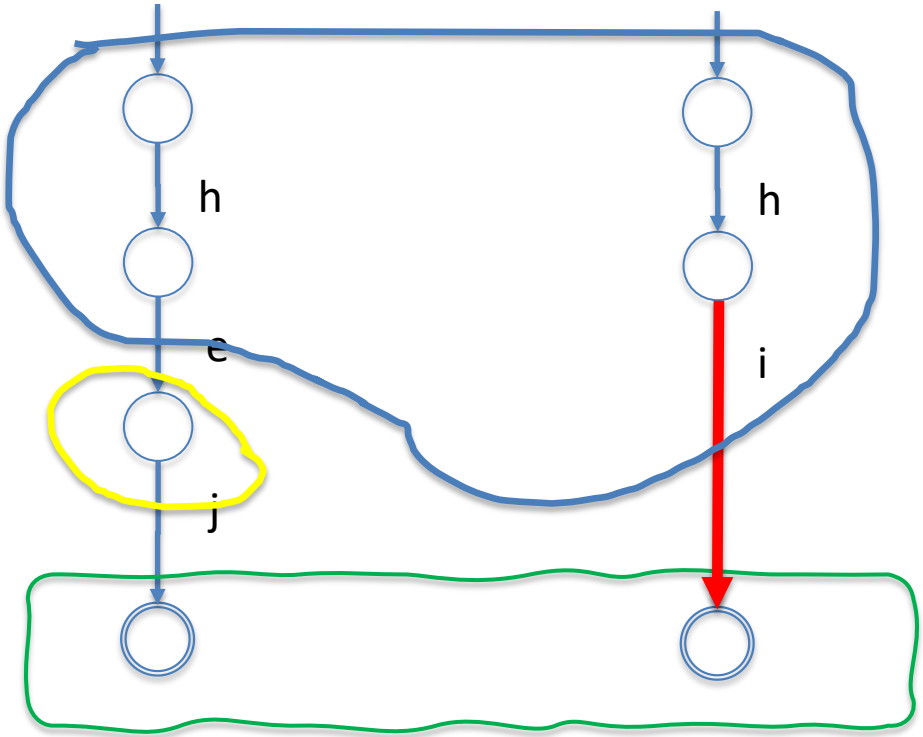
Challenge

Deciding (non) Equality of two DFA



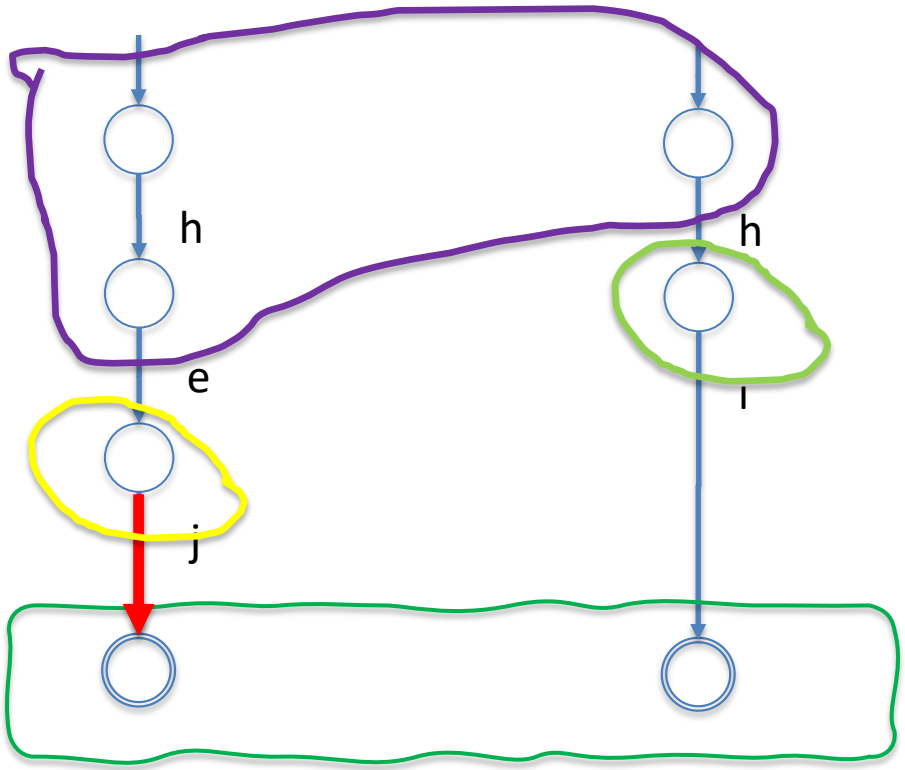
Spliting according to challenge

Deciding (non) Equality of two DFA



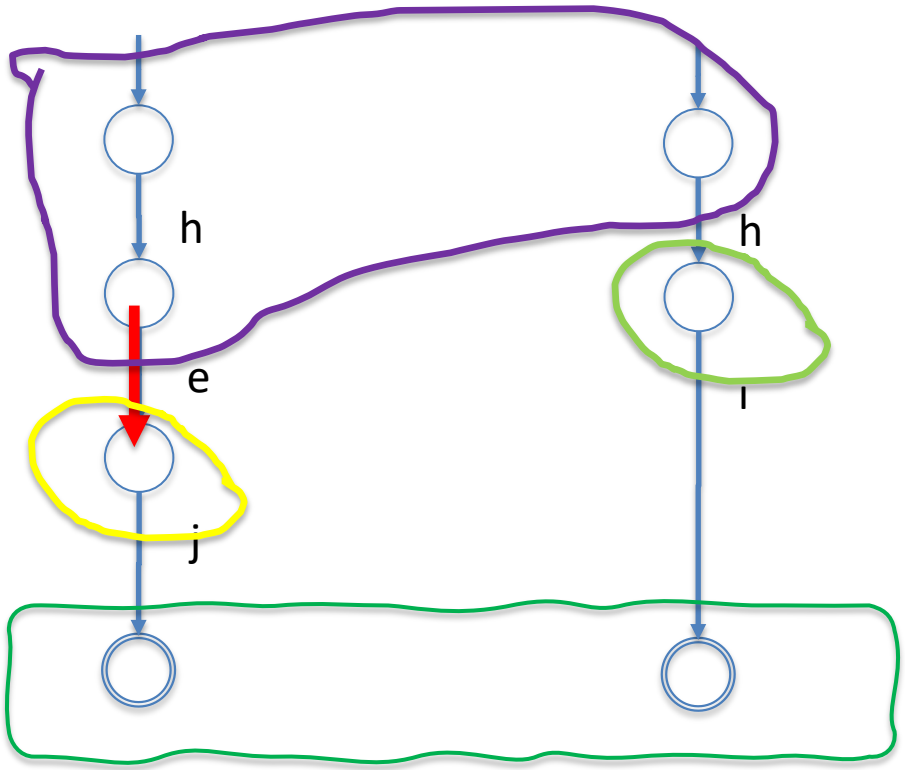
Challenge

Deciding (non) Equality of two DFA



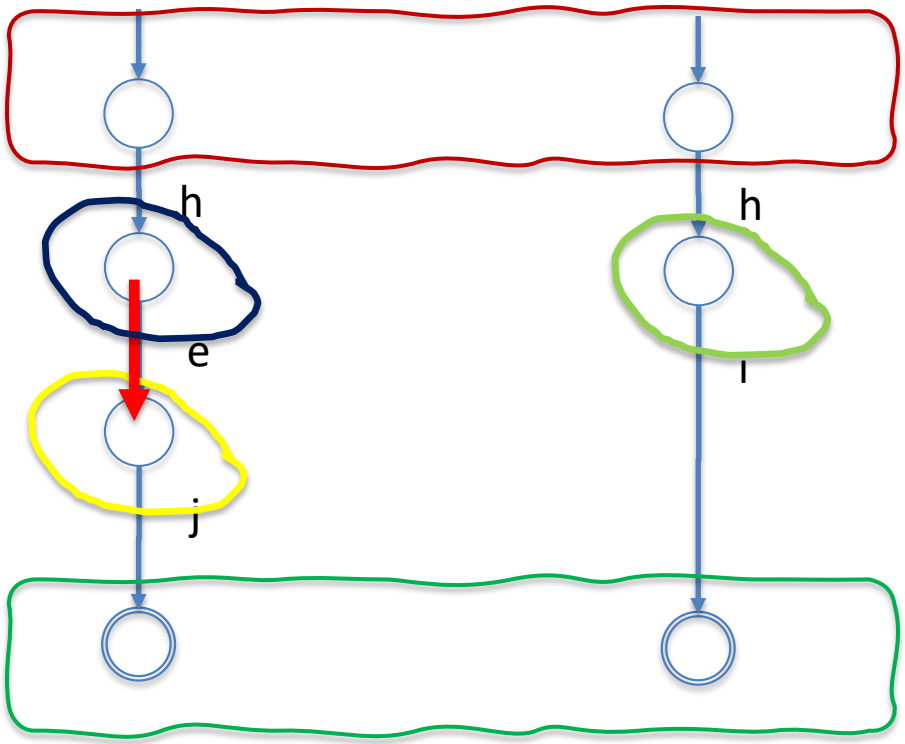
Spliting according to challenge

Deciding (non) Equality of two DFA



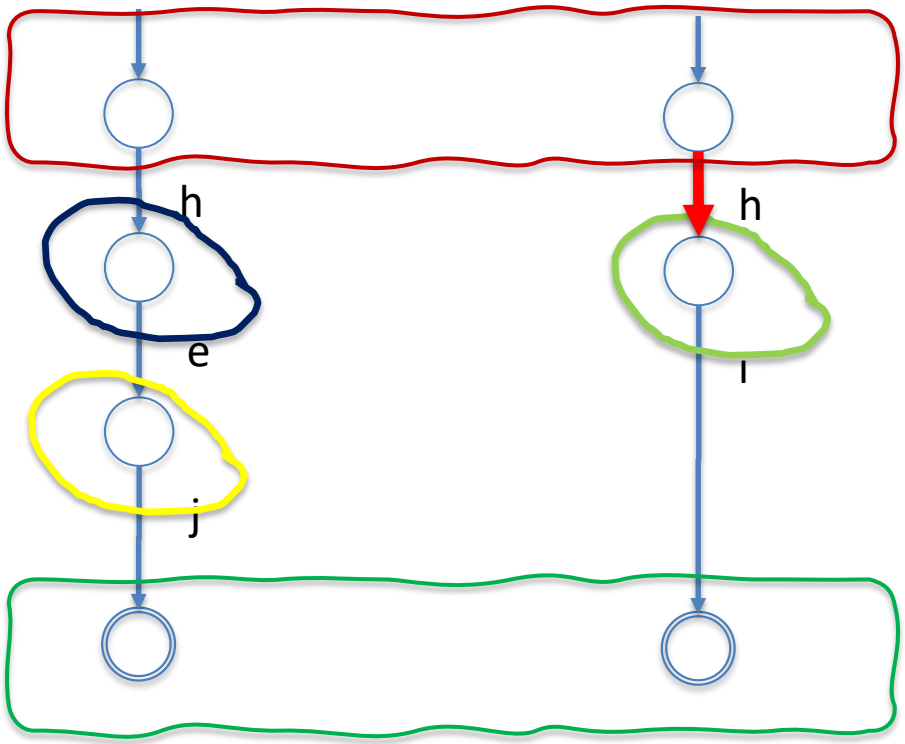
Challenge

Deciding (non) Equality of two DFA



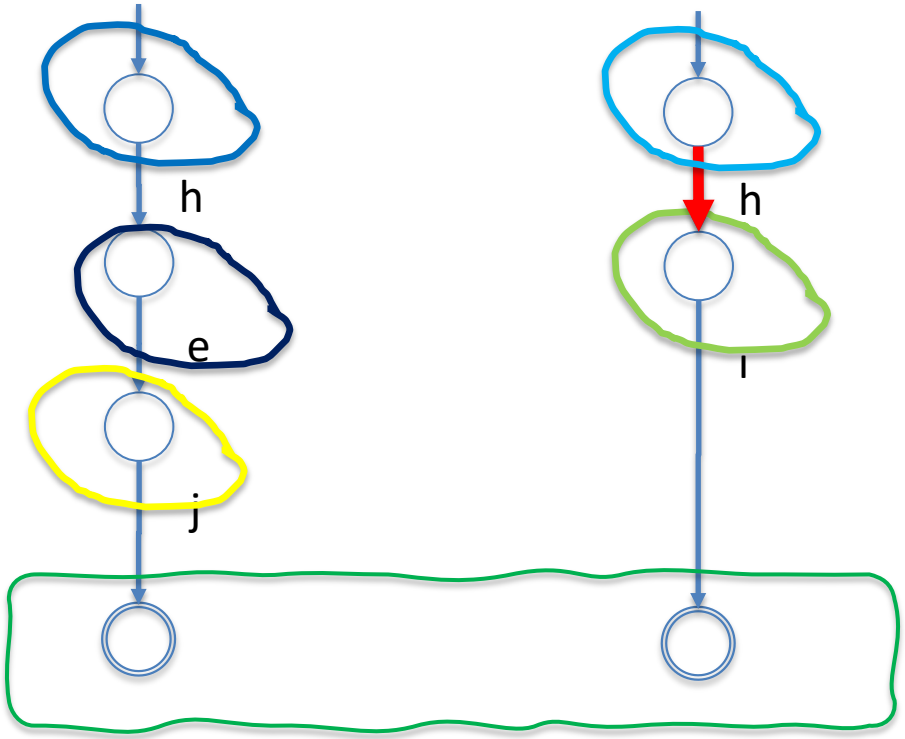
Spliting according to challenge

Deciding (non) Equality of two DFA



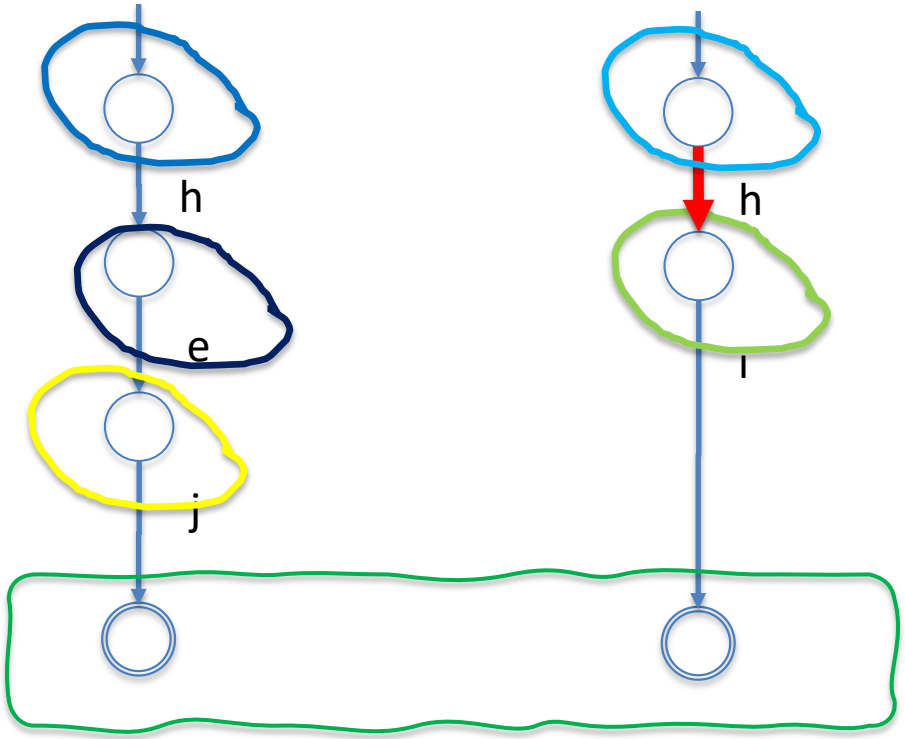
Challenge

Deciding (non) Equality of two DFA



Spliting according to challenge

Deciding (non) Equality of two DFA



No more challenges!

The two initial states are in differet groups

Hence, the DFA are not equivalent.

Deciding Equality (without DFA)

Example: Exercise RL1.6.